

รายงานฉบับเต็ม (Manual Report)

โครงการ: Retro Survivors Arena

1. บทนำและวัตถุประสงค์ (Introduction & Objectives)

1.1 ที่มาและความสำคัญ

เกมแนว Action-Survival ในยุคปัจจุบันได้รับความนิยมอย่างมากเนื่องจากเกมเพลย์ที่เข้าใจง่าย มีความท้าทายสูง และสามารถเล่นจบได้ในระยะเวลาอันสั้น โครงการนี้จึงนำแนวคิดเกมตู้ Arcade สไตล์ 8-bit มาพัฒนาร่วมกับไมโครคอนโทรลเลอร์ ESP32 เพื่อศึกษาการออกแบบระบบประมวลผลแบบ Real-time, การจัดการทรัพยากรหน่วยความจำที่มีจำกัด, และการสร้างอินเทอร์เฟซเชื่อมต่อระหว่างฮาร์ดแวร์กับซอฟต์แวร์อย่างเป็นระบบ

1.2 วัตถุประสงค์

- เพื่อพัฒนาระบบเกมตู้แบบ Embedded System บนไมโครคอนโทรลเลอร์ ESP32
- เพื่อประยุกต์ใช้แนวคิด Non-blocking Logic และ Finite State Machine (FSM) ในการควบคุมระบบ
- เพื่อเชื่อมต่อและควบคุมอุปกรณ์ต่อพ่วงหลากหลายรูปแบบ ได้แก่ จอ TFT LCD, Analog Joystick, Potentiometer, Push Button, Status LEDs และ Passive Buzzer

2. ข้อมูลทางเทคนิคและการออกแบบฮาร์ดแวร์ (Hardware Architecture)

2.1 รายการอุปกรณ์และการเชื่อมต่อ (Hardware Specifications & Pin Mapping)

ระบบใช้ไมโครคอนโทรลเลอร์ ESP32 DevKit เป็นแกนหลักในการประมวลผล โดยเชื่อมต่อกับอุปกรณ์ต่อพ่วงตามตารางดังนี้:

อุปกรณ์ต่อพ่วง (Component)	ขาฮาร์ดแวร์	ขา ESP32 (GPIO)	โหมดการทำงาน (Mode)	หน้าที่และความสำคัญ
Analog Joystick	VRx / VRy	GPIO 34 / GPIO 33	ADC Input (Analog)	อ่านค่าพิกัดแกน X/Y ควบคุมการเคลื่อนที่
Potentiometer	Wiper	GPIO 32	ADC Input (Analog)	อ่านค่าแรงดันเพื่อนำไปปรับความสว่าง Backlight
Push Button	Signal	GPIO 25	Digital Input (Pull-up)	ปุ่มเปิดใช้สกิลระเบิด (Skill) / ยืนยันเมนู
TFT Display Backlight	LED / BL	GPIO 14	PWM Output	รับสัญญาณ PWM ควบคุมความสว่างหน้าจอ
Status LEDs (3 สี)	Green/Yellow/Red	GPIO 16 / 17 / 5	Digital Output	แสดงสถานะพลังชีวิต (HP Bar) นอกหน้าจอ
Passive Piezo Buzzer	Pin (+)	GPIO 26	PWM / Tone Generator	ขับสัญญาณเสียงเอฟเฟกต์ SFX
TFT LCD (320x480)	SPI Pins	SPI Standard Pins	Hardware SPI	หน้าจอแสดงผลหลักของตัวเกม

2.2 การจัดการสัญญาณอนาล็อก (Analog Signal Handling)

- **Joystick Deadband Filter:** เนื่องจากค่าอนาล็อกที่อ่านได้จาก Joystick (0–4095) มักมีความผันผวนบริเวณจุดศูนย์กลาง จึงมีการตั้งค่า Deadband ไว้ที่ 1400–2600 หากค่าที่อ่านได้อยู่ในช่วงนี้ ระบบจะถือว่าไม่มีการโยกจอย
- **PWM Backlight Control:** อ่านค่าจาก Potentiometer บน GPIO 32 (0–4095) แล้วนำมาสเกลค่าเป็น 0–255 ส่งออกทาง PWM GPIO 14 เพื่อปรับระดับไฟ Backlight

3. การออกแบบซอฟต์แวร์ (Software Architecture)

3.1 สถานะการทำงานของเกม (Finite State Machine - FSM)

ระบบซอฟต์แวร์ถูกแบ่งการทำงานออกเป็น 4 สถานะหลัก ดังนี้:

1. **STATE_TITLE (หน้าจอเริ่มต้น):** แสดงหน้าจอ Title Screen แสดงชื่อเกม และคำแนะนำเริ่มต้น กดปุ่ม Skill เพื่อเริ่มเข้าสู่ตัวเกม
2. **STATE_PLAYING (หน้าจอเล่นเกม):**
 - อ่านค่า Joystick คำนวณตำแหน่งผู้เล่น
 - สุ่มสร้างศัตรู (Spawn Enemies) และอัปเดตตำแหน่งศัตรูวิ่งเข้าหาผู้เล่น
 - ตรวจสอบการโจมตีอัตโนมัติด้วยระยะแส้ (Whip Attack)
 - ตรวจสอบการชน (Collision Detection) ระหว่างศัตรูกับผู้เล่น
 - อัปเดตสถานะไฟ LED แสดง HP
3. **STATE_LEVELUP (หน้าจออัปเกรด):** หยุดการเคลื่อนที่ชั่วคราวเมื่อเก็บ Gem ครบตามกำหนด แสดงเมนูให้ผู้เล่นเลือกเพิ่มความเร็วการเคลื่อนที่ (Speed) หรือเพิ่มระยะการโจมตี (Whip Range)
4. **STATE_GAMEOVER (หน้าจอจบ):** เมื่อ HP ของผู้เล่นหมดลง แสดงผลคะแนนรวม (Score) และสถิติการรอดชีวิต

3.2 เทคนิค Non-blocking Logic

โค้ดทั้งหมดพัฒนาโดยไม่ใช้ฟังก์ชัน delay() เพื่อป้องกันไม่ให้งานของระบบชะงัก โดยใช้ฟังก์ชัน millis() ในการจับเวลาและกำหนดรอบการทำงาน (Task Scheduling) เช่น:

- การวาดเฟรมหน้าจอ (Frame Rate Control)
- การสุ่มสร้างศัตรูเป็นช่วงๆ
- การขับเสียงเอฟเฟกต์ Buzzer โดยไม่ขัดจังหวะการเรนเดอร์หลัก

4. คู่มือการใช้งานระบบ (User Manual)

4.1 การเตรียมการก่อนใช้งาน

1. ตรวจสอบการจ่ายไฟผ่านพอร์ต USB เข้ากับบอร์ด ESP32
2. ตรวจสอบหน้าจอ TFT LCD สว่างขึ้นและแสดงผลหน้าจอ Title Screen

4.2 ขั้นตอนการเล่นเกม

1. **การเริ่มเกม:** เมื่ออยู่ที่หน้า Title ให้กดปุ่ม **Push Button (Skill)** 1 ครั้ง เพื่อเข้าสู่การเล่น
2. **การบังคับตัวละคร:**
 - โยก Joystick ไปในทิศทางที่ต้องการเพื่อหลบหลีกศัตรู
 - ตัวละครจะทำการโจมตีศัตรูในระยะรอบตัวโดยอัตโนมัติ
3. **การใช้งานสกิลระเบิด (Bomb Skill):**
 - เมื่อสะสมเกจสกิลเต็ม บนหน้าจอจะแสดงข้อความ [BOMB]
 - กดปุ่ม Push Button เพื่อปล่อยสกิลระเบิดทำลายศัตรูทั้งหมดบนหน้าจอพร้อมกัน
4. **การเลือกอัปเกรด (Level Up):**
 - เมื่อหน้าจอ Level Up ปรากฏขึ้น โยก Joystick ขึ้น/ลง เพื่อเลือกลำดับหัวข้ออัปเกรด
 - กดปุ่ม Push Button เพื่อยืนยันตัวเลือก
5. **การปรับความสว่างหน้าจอ:** หมุนตัวหมุน Potentiometer เพื่อเพิ่มหรือลดระดับความสว่างตามต้องการ

4.3 ระบบไฟแสดงสถานะภายนอก (LED HP Status)

- ไฟสีเขียวติดสว่าง: ระดับ HP สูงกว่า 66%
- ไฟสีเหลืองติดสว่าง: ระดับ HP อยู่ระหว่าง 33%-66%
- ไฟสีแดงติดสว่าง/กระพริบ: ระดับ HP ต่ำกว่า 33% (ขั้นวิกฤต)

5. สรุปผลและการทดสอบ (Conclusion & Results)

จากการทดสอบชิ้นงานจริง พบว่าระบบประมวลผล ESP32 ร่วมกับโครงสร้างโค้ดแบบ FSM และ Non-blocking Logic สามารถขับเคลื่อนตัวเกมได้อย่างราบรื่น ไม่พบอาการค้างหรือความล่าช้าในการตอบสนอง ปุ่มกดและ Joystick ตอบสนองได้แม่นยำ ระบบไฟ LED และเสียงเอฟเฟกต์ตอบสนองตรงตามเหตุการณ์ในเกม ถือว่าบรรลุวัตถุประสงค์ของโครงงานทุกประการ